

LAPORAN PRAKTIKUM STRUKTUR DATA

Linked List



disusun Oleh:

Tiara Amalia Insani

NIM 2511532019

Dosen Pengampu : DR.Wahyudi, S.T, M.T

Asisten Praktikum : Jovantri Immanuel Gulo

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

KATA PENGANTAR

Puji syukur saya panjatkan kehadiran Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya saya dapat menyelesaikan laporan praktikum Struktur Data dengan baik dan tepat waktu. Laporan ini dibuat untuk memenuhi tugas praktikum mata kuliah Struktur Data dengan materi Linked List. Dalam praktikum ini, saya mempelajari cara membuat, menambah, mencari, menampilkan, dan menghapus data pada struktur data linked list menggunakan bahasa pemrograman Java. Selain itu, praktikum ini juga mengimplementasikan Singly Linked List pada sistem antrian rumah sakit. Saya menyadari bahwa dalam penyusunan laporan ini masih terdapat kekurangan. Oleh karena itu, saya mengharapkan kritik dan saran yang membangun agar laporan ini dapat menjadi lebih baik di masa mendatang.

Padang, 6 Mei 2026

Penyusun

DAFTAR ISI

Contents

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I.....	1
PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan Praktikum.....	1
1.3 Manfaat.....	1
BAB II	2
PEMBAHASAN	2
2.1 Pengertian LinkedList.....	2
2.2 Operasi dalam LinkedList.....	2
2.3 Praktikum	2
2.3.1 Alat dan Bahan.....	2
2.3.2 Program Class NodeSSL	3
2.3.3 Program Class TambahSSL	3
2.3.4 Program Class PencarianSSL	12
2.3.5 Program Class HapusSSL	15
BAB III.....	24
KESIMPULAN DAN SARAN	24
3.1 Kesimpulan.....	24
3.2 Saran	24
DAFTAR PUSTAKA.....	25
Penjelasan Tugas	26

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan salah satu bagian penting dalam pemrograman yang digunakan untuk mengatur dan menyimpan data agar dapat diproses dengan lebih efisien. Salah satu struktur data yang sering digunakan adalah Linked List. Linked List digunakan untuk menyimpan sekumpulan data yang saling terhubung melalui pointer atau referensi. Berbeda dengan array yang memiliki ukuran tetap, Linked List memiliki ukuran yang dinamis sehingga data dapat ditambah atau dihapus dengan lebih mudah. Pada Linked List, setiap node terdiri dari data dan pointer yang menunjuk ke node berikutnya. Dengan konsep tersebut, proses pengolahan data menjadi lebih fleksibel. Dalam praktikum ini dilakukan implementasi Linked List menggunakan bahasa pemrograman Java. Operasi yang dilakukan meliputi penambahan data, pencarian data, penghapusan data, serta penelusuran data pada Linked List.

1.2 Tujuan Praktikum

1. Memahami konsep dasar struktur data Linked List
2. Memahami cara kerja node dan pointer pada Linked List.
3. Mengimplementasikan operasi penambahan data pada Linked List.
4. Mengimplementasikan operasi pencarian data pada Linked List.
5. Mengimplementasikan operasi penghapusan data pada Linked List.

1.3 Manfaat

Praktikum ini bermanfaat untuk menambah pemahaman mengenai struktur data Linked List serta cara kerja node dan pointer dalam menghubungkan data. Melalui praktikum ini, mahasiswa dapat melatih kemampuan pemrograman menggunakan bahasa Java dalam melakukan operasi penambahan, pencarian, penelusuran, dan penghapusan data pada Linked List. Selain itu, praktikum ini juga membantu memahami pengelolaan data yang bersifat dinamis serta penerapan Linked List dalam kehidupan sehari-hari.

BAB II

PEMBAHASAN

2.1 Pengertian LinkedList

Linked List adalah struktur data linear yang terdiri dari sekumpulan node yang saling terhubung satu sama lain menggunakan pointer atau referensi. Setiap node pada Linked List memiliki dua bagian, yaitu data dan pointer yang menunjuk ke node berikutnya. Struktur data ini digunakan untuk menyimpan data secara dinamis sehingga ukuran data dapat bertambah atau berkurang sesuai kebutuhan.

Berbeda dengan array yang penyimpanan datanya harus berurutan di memori, Linked List menyimpan data secara tidak berurutan tetapi tetap terhubung melalui pointer. Karena itu, Linked List lebih fleksibel dalam proses penambahan dan penghapusan data.

2.2 Operasi dalam LinkedList

- a) Traversal → proses menelusuri atau menampilkan seluruh data pada Linked List dari node pertama hingga node terakhir.
- b) Insertion → proses menambahkan node baru ke dalam Linked List. Penambahan data dapat dilakukan di awal, di akhir, maupun di posisi tertentu.
- c) Deletion → proses menghapus node pada Linked List. Penghapusan dapat dilakukan pada node awal, node akhir, maupun node pada posisi tertentu.
- d) Searching → proses mencari data tertentu pada Linked List dengan cara menelusuri node satu per satu hingga data ditemukan.
- e) Updating → proses mengubah data yang terdapat pada node tertentu di dalam Linked List.

2.3 Praktikum

2.3.1 Alat dan Bahan

1. Laptop/computer dengan eclipse terinstal.
2. Materi praktikum mengenai Linked List

2.3.2 Program Class NodeSSL

```
package pekan5_25115320119;

public class NodeSSL_2511532019 {
    //node bagian data
    int data_2019;
    //pointer ke node berikutnya
    NodeSSL_2511532019 next_2019;
    //Konstruktor menginisialisasi node dengan data
    public NodeSSL_2511532019(int data_2019) {
        this.data_2019=data_2019;
        this.next_2019=null;
    }
}
```

1. `package pekan5_25115320119;`
kode berada dalam package Bernama pekan5_2511532019
2. `public class NodeSSL_2511532019 {`
baris ini mendeklarasikan class Bernama NodeSSL_2511532019 yang dapat diakses dari class lain karena menggunakan keyword public
3. `int data_2019;`
membuat variabel bertipe integer Bernama data_2019 untuk menyimpan nilai pada node.
4. `NodeSSL_2511532019 next_2019;`
Membuat variabel referensi Bernama next_2019 yang menunjuk ke node berikutnya
5. `public NodeSSL_2511532019(int data_2019) {`
baris ini mendeklarasikan konstruktor class NodeSSL_2511532019 yang menerima parameter data_2019
6. `this.data_2019=data_2019;`
mengisi variabel data_2019 milik objek dengan nilai parameter data_2019 yang diberikan saat objek dibuat
7. `this.next_2019=null;`
mengatur pointer next_2019 menjadi null, artinya node belum terhubung ke node lain

2.3.3 Program Class TambahSSL

```
package pekan5_25115320119;
```

```

public class TambahSSL_2511532019 {
    public static NodeSSL_2511532019 insertAtFront(NodeSSL_2511532019
head_2019, int value) {
        NodeSSL_2511532019 new_node_2019=new
NodeSSL_2511532019(value);
        new_node_2019.next_2019=head_2019;
        return new_node_2019;
    }
    //fungsi menambahkan node di akhir SSL
    public static NodeSSL_2511532019 insertAtEnd(NodeSSL_2511532019
head_2019, int value) {
        //buat sebuah node dengan sebuah nilai
        NodeSSL_2511532019 newNode_2019=new
NodeSSL_2511532019(value);
        //jika list kosong maka node menjadi head
        if (head_2019==null) {
            return newNode_2019;
        }
        //simpan head ke variabel sementara
        NodeSSL_2511532019 last_2019=head_2019;
        //telusuri ke node akhir
        while(last_2019.next_2019 !=null) {
            last_2019=last_2019.next_2019;
        }
        //ubah pointer
        last_2019.next_2019=newNode_2019;
        return head_2019;
    }
    static NodeSSL_2511532019 GetNode(int data_2019) {
        return new NodeSSL_2511532019(data_2019);
    }
    static NodeSSL_2511532019 insertPos(NodeSSL_2511532019
headNode_2019, int position_2019, int value) {
        NodeSSL_2511532019 head_2019=headNode_2019;
        if (position_2019<1)
            System.out.println("Invalid position");
        if(position_2019==1) {
            NodeSSL_2511532019 new_node_2019=new
NodeSSL_2511532019(value);
            new_node_2019.next_2019=head_2019;
            return new_node_2019;
        }else {
            while (position_2019-- != 0) {
                if (position_2019 == 1) {
                    NodeSSL_2511532019
newNode_2019=GetNode(value);

                    newNode_2019.next_2019=headNode_2019.next_2019;
                    headNode_2019.next_2019=newNode_2019;
                    break;
                }
                headNode_2019=headNode_2019.next_2019;
            }
            if(position_2019 !=1)
                System.out.println("Posisi di luar
jangkauan");
            return head_2019;
        }
    }
}

```

```

    }
}

public static void printList (NodeSSL_2511532019 head_2019)
{
    NodeSSL_2511532019 curr=head_2019;
    while(curr.next_2019 !=null) {
        System.out.print(curr.data_2019+"-->");
        curr=curr.next_2019;
    }
    if (curr.next_2019==null) {
        System.out.print(curr.data_2019);
    }
    System.out.println();
}

public static void main (String[] args) {
    //buat linked list 2->3->5->6
    NodeSSL_2511532019 head_2019=new
NodeSSL_2511532019(2);
    head_2019.next_2019=new NodeSSL_2511532019(3);
    head_2019.next_2019.next_2019=new
NodeSSL_2511532019(5);
    head_2019.next_2019.next_2019.next_2019=new
NodeSSL_2511532019(6);
    //cetak list asli
    System.out.print("Senarai berantai awal: ");
    printList(head_2019);
    //tambahkan node baru di depan
    System.out.print("tambah 1 simpul di depan: ");
    int data_2019=1;
    head_2019=insertAtFront(head_2019, data_2019);
    //cetak update list
    printList(head_2019);
    //tambahkan node baru di belakang
    System.out.print("tambah 1 simpul di belakang: ");
    int data2_2019=7;
    head_2019=insertAtEnd(head_2019, data2_2019);
    //cetak update list
    printList(head_2019);
    System.out.print("tambah 1 simpul di ke data 4: ");
    int data3_2019=4;
    int pos_2019=4;
    head_2019=insertPos(head_2019, pos_2019,
data3_2019);
    //cetak update list
    printList(head_2019);
}
}

```

```

public class TambahSSL_2511532019 {
    public static NodeSSL_2511532019 insertAtFront(NodeSSL_2511532019
head_2019, int value) {
        NodeSSL_2511532019 new_node_2019=new
NodeSSL_2511532019(value);
        new_node_2019.next_2019=head_2019;
        return new_node_2019;
    }
}

```



```
}
```

- `public class TambahSSL_2511532019 {`
→ Membuat class bernama `TambahSSL_2511532019`.
- `public static NodeSSL_2511532019`
`insertAtFront(NodeSSL_2511532019 head_2019, int value) {`
→ Membuat method `insertAtFront()` untuk menambahkan node di depan linked list.
- `NodeSSL_2511532019 new_node_2019=new`
`NodeSSL_2511532019(value);`
→ Membuat node baru dengan nilai `value`.
- `new_node_2019.next_2019=head_2019;`
→ Pointer node baru diarahkan ke head lama.
- `return new_node_2019;`
→ Mengembalikan node baru sebagai head baru.

```
//fungsi menambahkan node di akhir SSL
public static NodeSSL_2511532019 insertAtEnd(NodeSSL_2511532019
head_2019, int value) {
    //buat sebuah node dengan sebuah nilai
    NodeSSL_2511532019 newNode_2019=new
NodeSSL_2511532019(value);
```

- `public static NodeSSL_2511532019 insertAtEnd(NodeSSL_2511532019`
`head_2019, int value) {`
→ Membuat method `insertAtEnd()`.
- `NodeSSL_2511532019 newNode_2019=new`
`NodeSSL_2511532019(value);`
→ Membuat node baru dengan nilai tertentu.

```
//jika list kosong maka node menjadi head
if (head_2019==null) {
    return newNode_2019;
}
```

- `if (head_2019==null) {`
→ Mengecek apakah head kosong.
- `return newNode_2019;`
→ Jika kosong, node baru langsung menjadi head.

```
//simpan head ke variabel sementara
NodeSSL_2511532019 last_2019=head_2019;
//telusuri ke node akhir
while(last_2019.next_2019 !=null) {
    last_2019=last_2019.next_2019;
}
```

- NodeSSL_2511532019 last_2019=head_2019;
→Variabel last_2019 menyimpan posisi head.
- while(last_2019.next_2019 !=null) {
→Perulangan berjalan selama masih ada node berikutnya.
- last_2019=last_2019.next_2019;
→Pointer berpindah ke node berikutnya.

```
//ubah pointer
last_2019.next_2019=newNode_2019;
return head_2019;
}
```

- last_2019.next_2019=newNode_2019;
→Node terakhir diarahkan ke node baru.
- return head_2019;
→Mengembalikan head linked list.

```
static NodeSSL_2511532019 GetNode(int data_2019) {
    return new NodeSSL_2511532019(data_2019);
}
```

- static NodeSSL_2511532019 GetNode(int data_2019) {
→Membuat method GetNode().
- return new NodeSSL_2511532019(data_2019);
→Mengembalikan node baru dengan data tertentu.

```
static NodeSSL_2511532019 insertPos(NodeSSL_2511532019
headNode_2019, int position_2019, int value) {
    NodeSSL_2511532019 head_2019=headNode_2019;
    if (position_2019<1)
        System.out.println("Invalid position");
}
```

- static NodeSSL_2511532019 insertPos(...) {
→Membuat method untuk menyisipkan node pada posisi tertentu.
- NodeSSL_2511532019 head_2019=headNode_2019;
→Menyimpan head awal.
- if (position_2019<1)
→Mengecek apakah posisi kurang dari 1.

- `System.out.println("Invalid position");`
→ Menampilkan pesan posisi tidak valid.

```
if(position_2019==1) {
    NodeSSL_2511532019 new_node_2019=new NodeSSL_2511532019(value);
    new_node_2019.next_2019=head_2019;
    return new_node_2019;
}
```

- `if(position_2019==1) {`
→ Mengecek apakah posisi di awal.
- `NodeSSL_2511532019 new_node_2019=new`
`NodeSSL_2511532019(value);`
→ Membuat node baru.
- `new_node_2019.next_2019=head_2019;`
→ Node baru diarahkan ke head lama.
- `return new_node_2019;`
→ Node baru dijadikan head baru.

```
}else {
    while (position_2019-- != 0) {
        if (position_2019 == 1) {
            NodeSSL_2511532019 newNode_2019=GetNode(value);
```

- `}else {`
→ Masuk ke kondisi selain posisi pertama.
- `while (position_2019-- != 0) {`
→ Perulangan mencari posisi tujuan.
- `if (position_2019 == 1) {`
→ Mengecek apakah posisi sudah sesuai.
- `NodeSSL_2511532019 newNode_2019=GetNode(value);`
→ Membuat node baru.

```
newNode_2019.next_2019=headNode_2019.next_2019;
headNode_2019.next_2019=newNode_2019;
break;
}
```

- `newNode_2019.next_2019=headNode_2019.next_2019;`
→ Pointer node baru diarahkan ke node berikutnya.
- `headNode_2019.next_2019=newNode_2019;`
→ Pointer node sebelumnya diarahkan ke node baru.

- break;
→Menghentikan perulangan.

```
        headNode_2019=headNode_2019.next_2019;
    }
    if(position_2019 !=1)
        System.out.println("Posisi di luar jangkauan");
    return head_2019;
}
```

- headNode_2019=headNode_2019.next_2019;
→Pointer berpindah ke node berikutnya.
- if(position_2019 !=1)
→Mengecek apakah posisi tidak ditemukan.
- System.out.println("Posisi di luar jangkauan");
→Menampilkan pesan error posisi.
- return head_2019;
→Mengembalikan head linked list.

```
public static void printList (NodeSSL_2511532019 head_2019) {
    NodeSSL_2511532019 curr=head_2019;
    while(curr.next_2019 !=null) {
        System.out.print(curr.data_2019+"-->");
    }
}
```

- public static void printList (...) {
→Membuat method untuk mencetak linked list.
- NodeSSL_2511532019 curr=head_2019;
→Variabel curr digunakan untuk traversal.
- while(curr.next_2019 !=null) {
→Perulangan selama masih ada node berikutnya.
- System.out.print(curr.data_2019+"-->");
→Mencetak data node.

```
        curr=curr.next_2019;
    }
    if (curr.next_2019==null) {
        System.out.print(curr.data_2019);
    }
    System.out.println();
}
```

- curr=curr.next_2019;
→Pointer berpindah ke node berikutnya.

- `if (curr.next_2019==null) {`
→ Mengecek node terakhir.
- `System.out.print(curr.data_2019);`
→ Mencetak data node terakhir.
- `System.out.println();`
→ Mencetak baris baru.

```
public static void main (String[] args) {
    //buat linked list 2->3->5->6
    NodeSSL_2511532019 head_2019=new
NodeSSL_2511532019(2);
    head_2019.next_2019=new NodeSSL_2511532019(3);
    head_2019.next_2019.next_2019=new
NodeSSL_2511532019(5);
    head_2019.next_2019.next_2019.next_2019=new
NodeSSL_2511532019(6);
}
```

- `public static void main (String[] args) {`
→ Method utama program.
- `NodeSSL_2511532019 head_2019=new NodeSSL_2511532019(2);`
→ Membuat node pertama bernilai 2.
- `head_2019.next_2019=new NodeSSL_2511532019(3);`
→ Menambahkan node bernilai 3.
- `head_2019.next_2019.next_2019=new NodeSSL_2511532019(5);`
→ Menambahkan node bernilai 5.
- `head_2019.next_2019.next_2019.next_2019=new`
`NodeSSL_2511532019(6);`
→ Menambahkan node bernilai 6.

```
//cetak list asli
System.out.print("Senarai berantai awal: ");
printList(head_2019);
```

- `System.out.print("Senarai berantai awal: ");`
→ Menampilkan teks awal.
- `printList(head_2019);`
→ Memanggil method mencetak list.

```
//tambahkan node baru di depan
System.out.print("tambah 1 simpul di depan: ");
int data_2019=1;
head_2019=insertAtFront(head_2019, data_2019);
```

- `System.out.print("tambah 1 simpul di depan: ");`
→ Menampilkan teks.
- `int data_2019=1;`
→ Membuat data bernilai 1.
- `head_2019=insertAtFront(head_2019, data_2019);`
→ Menambahkan node di depan.

```
//cetak update list
printList(head_2019);
//tambahkan node baru di belakang
System.out.print("tambah 1 simpul di belakang: ");
int data2_2019=7;
head_2019=insertAtEnd(head_2019, data2_2019);
```

- `printList(head_2019);`
→ Mencetak linked list.
- `System.out.print("tambah 1 simpul di belakang: ");`
→ Menampilkan teks.
- `int data2_2019=7;`
→ Membuat data bernilai 7.
- `head_2019=insertAtEnd(head_2019, data2_2019);`
→ Menambahkan node di belakang.

```
//cetak update list
printList(head_2019);
System.out.print("tambah 1 simpul di ke data 4: ");
int data3_2019=4;
int pos_2019=4;
head_2019=insertPos(head_2019, pos_2019, data3_2019);
//cetak update list
printList(head_2019);
}
```

- `printList(head_2019);`
→ Mencetak linked list terbaru.
- `System.out.print("tambah 1 simpul di ke data 4: ");`
→ Menampilkan teks penambahan node.
- `int data3_2019=4;`
→ Data node bernilai 4.
- `int pos_2019=4;`
→ Posisi penyisipan adalah posisi ke-4.

- head_2019=insertPos(head_2019, pos_2019, data3_2019);
→Menyisipkan node pada posisi tertentu.
- printList(head_2019);
→Mencetak linked list akhir.

Output

```
Senarai berantai awal: 2-->3-->5-->6
tambah 1 simpul di depan: 1-->2-->3-->5-->6
tambah 1 simpul di belakang: 1-->2-->3-->5-->6-->7
tambah 1 simpul di ke data 4: 1-->2-->3-->4-->5-->6-->7
```

2.3.4 Program Class PencarianSSL

```
package pekan5_25115320119;

public class PencarianSSL_2511532019 {
    static boolean searchKey(NodeSSL_2511532019 head_2019, int key) {
        NodeSSL_2511532019 curr_2019 = head_2019;
        while(curr_2019 !=null) {
            if (curr_2019.data_2019 == key)
                return true;
            curr_2019 = curr_2019.next_2019;}
        return false;}
    public static void traversal (NodeSSL_2511532019 head_2019) {
        //mulai dari head
        NodeSSL_2511532019 curr_2019 = head_2019;
        //telusuri sampai pointer null
        while (curr_2019!=null) {
            System.out.print(" "+curr_2019.data_2019);
            curr_2019=curr_2019.next_2019;}
        System.out.println();}
    public static void main (String[] args) {
        NodeSSL_2511532019 head_2019 = new NodeSSL_2511532019(14);
        head_2019.next_2019=new NodeSSL_2511532019(21);
        head_2019.next_2019.next_2019=new NodeSSL_2511532019(13);
        head_2019.next_2019.next_2019.next_2019=new
NodeSSL_2511532019(30);
        head_2019.next_2019.next_2019.next_2019.next_2019=new
NodeSSL_2511532019(10);
        System.out.print("Penelusuran SSL : ");
        traversal (head_2019);
        //data yang akan dicari
        int key=30;
        System.out.print("cari data "+key+" = ");
        if (searchKey(head_2019, key))
            System.out.println("ketemu");
        else
            System.out.println("tidak ada");
    }
}
```

```
public class PencarianSSL_2511532019 {
    static boolean searchKey(NodeSSL_2511532019 head_2019, int key) {
        NodeSSL_2511532019 curr_2019 = head_2019;
```

- public class PencarianSSL_2511532019 {
→Membuat class bernama PencarianSSL_2511532019.
- static boolean searchKey(NodeSSL_2511532019 head_2019, int key) {
→Membuat method searchKey() untuk mencari data pada linked list.
- NodeSSL_2511532019 curr_2019 = head_2019;
→Variabel curr_2019 digunakan untuk traversal mulai dari head.

```
    while(curr_2019 !=null) {
        if (curr_2019.data_2019 == key)
            return true;
        curr_2019 = curr_2019.next_2019;}
    return false;}
```

- while(curr_2019 !=null) {
→Perulangan berjalan selama node masih ada.
- if (curr_2019.data_2019 == key)
→Mengecek apakah data node sama dengan nilai yang dicari.
- return true;
→Jika data ditemukan, method mengembalikan nilai true.
- curr_2019 = curr_2019.next_2019;
→Pointer berpindah ke node berikutnya.
- return false;
→Jika data tidak ditemukan, method mengembalikan false.

```
public static void traversal (NodeSSL_2511532019 head_2019) {
    //mulai dari head
    NodeSSL_2511532019 curr_2019 = head_2019;
    //telusuri sampai pointer null
    while (curr_2019!=null) {
        System.out.print(" "+curr_2019.data_2019);
        curr_2019=curr_2019.next_2019;}
    System.out.println();}
```

- public static void traversal (NodeSSL_2511532019 head_2019) {
Membuat method traversal() untuk menampilkan isi linked list.

- NodeSSL_2511532019 curr_2019 = head_2019;
→ Variabel curr_2019 digunakan untuk traversal.
- while (curr_2019!=null) {
→ Perulangan berjalan selama node masih ada.
- System.out.print(" "+curr_2019.data_2019);
→ Menampilkan data setiap node.
- curr_2019=curr_2019.next_2019;
→ Pointer berpindah ke node berikutnya.
- System.out.println();
→ Membuat baris baru setelah traversal selesai.

```
public static void main (String[] args) {
    NodeSSL_2511532019 head_2019 = new NodeSSL_2511532019(14);
    head_2019.next_2019=new NodeSSL_2511532019(21);
    head_2019.next_2019.next_2019=new NodeSSL_2511532019(13);
    head_2019.next_2019.next_2019.next_2019=new
NodeSSL_2511532019(30);
    head_2019.next_2019.next_2019.next_2019.next_2019=new
NodeSSL_2511532019(10);
    System.out.print("Penelusuran SSL : ");
    traversal (head_2019);
}
```

- public static void main (String[] args) {
→ Method utama program.
- NodeSSL_2511532019 head_2019 = new NodeSSL_2511532019(14);
→ Membuat node pertama bernilai 14.
- head_2019.next_2019=new NodeSSL_2511532019(21);
→ Menambahkan node bernilai 21.
- head_2019.next_2019.next_2019=new NodeSSL_2511532019(13);
→ Menambahkan node bernilai 13.
- head_2019.next_2019.next_2019.next_2019=new
NodeSSL_2511532019(30);
→ Menambahkan node bernilai 30.
- head_2019.next_2019.next_2019.next_2019.next_2019=new
NodeSSL_2511532019(10);
→ Menambahkan node bernilai 10.

- `System.out.print("Penelusuran SSL : ");`
→ Menampilkan teks penelusuran linked list.
- `traversal (head_2019);`
→ Memanggil method traversal untuk mencetak isi linked list.

```
//data yang akan dicari
int key=30;
System.out.print("cari data "+key+" = ");
if (searchKey(head_2019, key))
    System.out.println("ketemu");
else
    System.out.println("tidak ada");
}
```

- `int key=30;`
→ Variabel key berisi nilai 30.
- `System.out.print("cari data "+key+" = ");`
→ Menampilkan teks pencarian data.
- `if (searchKey(head_2019, key))`
→ Mengecek apakah data ditemukan menggunakan method `searchKey()`.
- `System.out.println("ketemu");`
→ Ditampilkan jika data ditemukan.
- `else`
→ Kondisi jika data tidak ditemukan.
- `System.out.println("tidak ada");`
→ Ditampilkan jika data tidak ada.

Output

```
Penelusuran SSL :  14 21 13 30 10
cari data 30 = ketemu
```

2.3.5 Program Class HapusSSL

```
package pekan5_25115320119;

public class HapusSSL_2511532019 {
    //fungsi untuk menghapus head
    public static NodeSSL_2511532019
deleteHead_2019(NodeSSL_2511532019 head_2019) {
        //jika SSL kosong
        if (head_2019 == null)
            return null;
        //pindahkan head ke node berikutnya
    }
}
```

```

        head_2019=head_2019.next_2019;
        //return head baru
        return head_2019; }
    //fungsi menghapus node terakhir SSL
    public static NodeSSL_2511532019
removeLastNode_2019(NodeSSL_2511532019 head_2019) {
    //jika list kosong, return null
    if (head_2019==null) {
        return null;
    }
    //jika list satu node, hapus node dan return null
    if (head_2019.next_2019==null) {
        return null;
    }
    //temukan node terakhir ke dua
    NodeSSL_2511532019 secondLast_2019=head_2019;
    while(secondLast_2019.next_2019.next_2019 != null) {
        secondLast_2019=secondLast_2019.next_2019;
    }
    //hapus node terakhir
    secondLast_2019.next_2019=null;
    return head_2019;
}
    //fungsi menghapus node di posisi tertentu
    public static NodeSSL_2511532019
deleteNode_2019(NodeSSL_2511532019 head_2019, int position) {
    NodeSSL_2511532019 temp_2019=head_2019;
    NodeSSL_2511532019 prev_2019=null;
    //jika linked list null
    if (temp_2019==null)
        return head_2019;
    //kasus 1:head dihapus
    if (position == 1) {
        head_2019=temp_2019.next_2019;
        return head_2019;}
    //kasus 2: menghapus node di tengah
    //telusuri ke node yang dihapus
    for (int i=1;temp_2019!=null&&i<position;i++) {
        prev_2019=temp_2019;
        temp_2019=temp_2019.next_2019;}
    //jika ditemukan, hapus node
    if (temp_2019 != null) {
        prev_2019.next_2019=temp_2019.next_2019;
    }else {
        System.out.println("Data tidak ada");
    }
    return head_2019;}

    //fungsi mencetak SSL
    public static void printList_2019 (NodeSSL_2511532019 head_2019)
{
    NodeSSL_2511532019 curr=head_2019;
    while(curr.next_2019 != null) {
        System.out.print(curr.data_2019+"-->");
        curr = curr.next_2019;}
    if (curr.next_2019==null) {
        System.out.print(curr.data_2019); }
}

```

```

        System.out.println();}

//kelas main
public static void main (String[] args) {
    //buat SSL 1->2->3->4->5->6->null
    NodeSSL_2511532019 head_2019=new NodeSSL_2511532019(1);
    head_2019.next_2019=new NodeSSL_2511532019(2);
    head_2019.next_2019.next_2019=new NodeSSL_2511532019(3);
    head_2019.next_2019.next_2019.next_2019=new
NodeSSL_2511532019(4);
    head_2019.next_2019.next_2019.next_2019.next_2019=new
NodeSSL_2511532019(5);

    head_2019.next_2019.next_2019.next_2019.next_2019.next_2019=new
NodeSSL_2511532019(6);
    //cetak list awal
    System.out.println("list awal :");
    printList_2019(head_2019);
    //hapus head
    head_2019=deleteHead_2019(head_2019);
    System.out.println("List setelah head dihapus:");
    printList_2019(head_2019);
    //hapus node terakhir
    head_2019=removeLastNode_2019(head_2019);
    System.out.println("List setelah simpul terakhir dihapus:
");

    printList_2019(head_2019);
    //deleting node at position 2
    int position_2019 = 2;
    head_2019=deleteNode_2019(head_2019, position_2019);
    //print list after deletion
    System.out.println("List setelah posisi 2 dihapus :");
    printList_2019(head_2019);
}
}

```

```

public class HapusSSL_2511532019 {
    //fungsi untuk menghapus head
    public static NodeSSL_2511532019
deleteHead_2019(NodeSSL_2511532019 head_2019) {
        //jika SSL kosong
        if (head_2019 == null)
            return null;
        //pindahkan head ke node berikutnya
        head_2019=head_2019.next_2019;
        //return head baru
        return head_2019; }
}

```

- `public class HapusSSL_2511532019 {`
→ Membuat class bernama HapusSSL_2511532019.
- `public static NodeSSL_2511532019`
`deleteHead_2019(NodeSSL_2511532019 head_2019) {`
→ Membuat method `deleteHead_2019()` untuk menghapus node pertama.
pertama.
- `if (head_2019 == null)`
→ Mengecek apakah linked list kosong.
- `return null;`
→ Jika kosong, method mengembalikan null.
- `head_2019=head_2019.next_2019;`
→ Head dipindahkan ke node berikutnya.
- `return head_2019;`
→ Mengembalikan head yang baru.

```
//fungsi menghapus node terakhir SSL
public static NodeSSL_2511532019
removeLastNode_2019(NodeSSL_2511532019 head_2019) {
    //jika list kosong, return null
    if (head_2019==null) {
        return null;
    }
    //jika list satu node, hapus node dan return null
    if (head_2019.next_2019==null) {
        return null;
    }
    //temukan node terakhir ke dua
    NodeSSL_2511532019 secondLast_2019=head_2019;
    while(secondLast_2019.next_2019.next_2019 != null) {
        secondLast_2019=secondLast_2019.next_2019;
    }
    //hapus node terakhir
    secondLast_2019.next_2019=null;
    return head_2019;
}
```

- `public static NodeSSL_2511532019 removeLastNode_2019(...) {`
→ Membuat method `removeLastNode_2019()`.
- `if (head_2019==null) {`
→ Mengecek apakah list kosong.
- `return null;`
→ Jika list kosong, kembalikan null.

- `if (head_2019.next_2019==null) {`
→ Mengecek apakah hanya ada satu node.
- `return null;`
→ Menghapus node dengan mengembalikan null.
- `NodeSSL_2511532019 secondLast_2019=head_2019;`
→ Variabel `secondLast_2019` mulai dari head.
- `while(secondLast_2019.next_2019.next_2019 != null) {`
→ Perulangan berjalan sampai node kedua terakhir ditemukan.
- `secondLast_2019=secondLast_2019.next_2019;`
→ Pointer berpindah ke node berikutnya.
- `secondLast_2019.next_2019=null;`
→ Pointer node kedua terakhir dibuat null.
- `return head_2019;`
→ Mengembalikan head linked list.

```
//fungsi menghapus node di posisi tertentu
public static NodeSSL_2511532019
deleteNode_2019(NodeSSL_2511532019 head_2019, int position) {
    NodeSSL_2511532019 temp_2019=head_2019;
    NodeSSL_2511532019 prev_2019=null;
    //jika linked list null
    if (temp_2019==null)
        return head_2019;
```

- `public static NodeSSL_2511532019 deleteNode_2019(...) {`
→ Membuat method `deleteNode_2019()`.
- `NodeSSL_2511532019 temp_2019=head_2019;`
→ Variabel sementara untuk traversal.
- `NodeSSL_2511532019 prev_2019=null;`
→ Variabel untuk menyimpan node sebelumnya.
- `if (temp_2019==null)`
→ Mengecek apakah list kosong.
- `return head_2019;`
→ Jika kosong, kembalikan head.

```
//kasus 1:head dihapus
if (position == 1) {
    head_2019=temp_2019.next_2019;
```

```

        return head_2019;}
        //kasus 2: menghapus node di tengah
        //telusuri ke node yang dihapus
        for (int i=1;temp_2019!=null&&i<position;i++) {
            prev_2019=temp_2019;
            temp_2019=temp_2019.next_2019;}
        //jika ditemukan, hapus node
        if (temp_2019 != null) {
            prev_2019.next_2019=temp_2019.next_2019;
        }else {
            System.out.println("Data tidak ada");
        }
        return head_2019;}

```

- if (position == 1) {
→Mengecek apakah posisi yang dihapus adalah posisi pertama.
- head_2019=temp_2019.next_2019;
→Head dipindahkan ke node berikutnya.
- return head_2019;
→Mengembalikan head baru.
- for (int i=1;temp_2019!=null&&i<position;i++) {
→Perulangan menuju posisi node yang ingin dihapus.
- prev_2019=temp_2019;
→Menyimpan node sebelumnya.
- temp_2019=temp_2019.next_2019;
→Pointer berpindah ke node berikutnya.
- if (temp_2019 != null) {
→Mengecek apakah node ditemukan.
- prev_2019.next_2019=temp_2019.next_2019;
→Node sebelumnya diarahkan ke node setelah node yang dihapus.
- }else {
→Kondisi jika node tidak ditemukan.
- System.out.println("Data tidak ada");
→Menampilkan pesan error.
- return head_2019;
Mengembalikan head linked list.

```

//fungsi mencetak SSL

```

```

public static void printList_2019 (NodeSSL_2511532019 head_2019)
{
    NodeSSL_2511532019 curr=head_2019;
    while(curr.next_2019 != null) {
        System.out.print(curr.data_2019+"-->");
        curr = curr.next_2019;}
    if (curr.next_2019==null) {
        System.out.print(curr.data_2019); }
    System.out.println();}

```

- public static void printList_2019 (...) {
→ Membuat method printList_2019().
- NodeSSL_2511532019 curr=head_2019;
→ Variabel curr digunakan untuk traversal.
- while(curr.next_2019 != null) {
→ Perulangan selama masih ada node berikutnya.
- System.out.print(curr.data_2019+"-->");
→ Menampilkan data node.
- curr = curr.next_2019;
→ Pointer berpindah ke node berikutnya.
- if (curr.next_2019==null) {
→ Mengecek node terakhir.
- System.out.print(curr.data_2019);
→ Menampilkan data node terakhir.
- System.out.println();
Membuat baris baru.

```

//kelas main
public static void main (String[] args) {
    //buat SSL 1->2->3->4->5->6->null
    NodeSSL_2511532019 head_2019=new NodeSSL_2511532019(1);
    head_2019.next_2019=new NodeSSL_2511532019(2);
    head_2019.next_2019.next_2019=new NodeSSL_2511532019(3);
    head_2019.next_2019.next_2019.next_2019=new
NodeSSL_2511532019(4);
    head_2019.next_2019.next_2019.next_2019.next_2019=new
NodeSSL_2511532019(5);

    head_2019.next_2019.next_2019.next_2019.next_2019.next_2019=new
NodeSSL_2511532019(6);
}

```

- public static void main (String[] args) {
→ Method utama program.

- NodeSSL_2511532019 head_2019=new NodeSSL_2511532019(1);
→ Membuat node pertama bernilai 1.
- head_2019.next_2019=new NodeSSL_2511532019(2);
→ Menambahkan node bernilai 2.
- head_2019.next_2019.next_2019=new NodeSSL_2511532019(3);
→ Menambahkan node bernilai 3.
- head_2019.next_2019.next_2019.next_2019=new --
NodeSSL_2511532019(4);
→ Menambahkan node bernilai 4.
- head_2019.next_2019.next_2019.next_2019.next_2019=new
NodeSSL_2511532019(5);
→ Menambahkan node bernilai 5.
- head_2019.next_2019.next_2019.next_2019.next_2019.next_2019=new
NodeSSL_2511532019(6);
→ Menambahkan node bernilai 6.

```
//cetak list awal
System.out.println("list awal :");
printList_2019(head_2019);
```

- System.out.println("list awal :");
→ Menampilkan teks list awal.
- printList_2019(head_2019);
→ Mencetak linked list.

```
//hapus head
head_2019=deleteHead_2019(head_2019);
System.out.println("List setelah head dihapus:");
printList_2019(head_2019);
```

- head_2019=deleteHead_2019(head_2019);
→ Menghapus node pertama.
- System.out.println("List setelah head dihapus:");
→ Menampilkan teks hasil penghapusan.
- printList_2019(head_2019);
→ Mencetak linked list terbaru.

```
//hapus node terakhir
```

```

        head_2019=removeLastNode_2019(head_2019);
        System.out.println("List setelah simpul terakhir dihapus:
");
        printList_2019(head_2019);

```

- head_2019=removeLastNode_2019(head_2019);
→Menghapus node terakhir.
- System.out.println("List setelah simpul terakhir dihapus: ");
→Menampilkan teks hasil penghapusan.
- printList_2019(head_2019);
→Mencetak linked list terbaru.

```

//deleting node at position 2
int position_2019 = 2;
head_2019=deleteNode_2019(head_2019, position_2019);

```

- int position_2019 = 2;
→Menyimpan posisi yang akan dihapus.
- head_2019=deleteNode_2019(head_2019, position_2019);
→Menghapus node pada posisi tersebut.

```

//print list after deletion
System.out.println("List setelah posisi 2 dihapus :");
printList_2019(head_2019);
}

```

- System.out.println("List setelah posisi 2 dihapus :");
→Menampilkan teks hasil akhir.
- printList_2019(head_2019);
→Mencetak linked list setelah penghapusan.

Output

```

list awal :
1-->2-->3-->4-->5-->6
List setelah head dihapus:
2-->3-->4-->5-->6
List setelah simpul terakhir dihapus:
2-->3-->4-->5
List setelah posisi 2 dihapus :
2-->4-->5

```

BAB III

KESIMPULAN DAN SARAN

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Linked List merupakan struktur data linear yang terdiri dari node-node yang saling terhubung menggunakan pointer atau referensi. Linked List memiliki sifat dinamis sehingga memudahkan proses penambahan dan penghapusan data dibandingkan array. Pada praktikum ini telah dilakukan beberapa operasi pada Linked List, yaitu traversal, insertion, deletion, dan searching menggunakan bahasa pemrograman Java. Dengan praktikum ini, pemahaman mengenai cara kerja node, pointer, dan pengelolaan data pada Linked List menjadi lebih baik.

3.2 Saran

1. lebih banyak berlatih dalam mengimplementasikan LinkedList agar semakin memahami konsepnya
2. Perlu meningkatkan ketelitian dalam penulisan kode untuk menghindari kesalahan (*error*) saat program dijalankan.

DAFTAR PUSTAKA

- [1] M. T. Goodrich, R. Tamassia, dan M. H. Goldwasser, *Data Structures and Algorithms in Java*, 6th ed. Hoboken, NJ: Wiley, 2014.

Penjelasan Tugas

Program Class Pasien

```
package pekan5_25115320119;

public class Pasien_2511532019 {
    //atribut pasien
    String namaPasien_2019;
    String keluhan_2019;
    int nomorAntrian_2019;

    //pointer ke pasien berikutnya
    Pasien_2511532019 next_2019;

    public Pasien_2511532019(String namaPasien_2019, String
keluhan_2019, int nomorAntrian_2019) {
        this.namaPasien_2019=namaPasien_2019;
        this.keluhan_2019=keluhan_2019;
        this.nomorAntrian_2019=nomorAntrian_2019;
        this.next_2019=null;
    }

    public String getNamaPasien_2019() {
        return namaPasien_2019;
    }
    public String getKeluhan_2019() {
        return keluhan_2019;
    }
    public int getNomorAntrian_2019() {
        return nomorAntrian_2019;
    }
    public Pasien_2511532019 getNext_2019() {
        return next_2019;
    }

    public void setNamaPasien_2019(String namaPasien_2019) {
        this.namaPasien_2019=namaPasien_2019;
    }
    public void setKeluhan_2019(String keluhan_2019) {
        this.keluhan_2019=keluhan_2019;
    }
    public void setNomorAntrian_2019(int nomorAntrian_2019) {
        this.nomorAntrian_2019=nomorAntrian_2019;
    }
    public void setNext_2019(Pasien_2511532019 next_2019) {
        this.next_2019=next_2019;
    }
}
```

```
public class Pasien_2511532019 {
    //atribut pasien
```

```
String namaPasien_2019;
String keluhan_2019;
int nomorAntrian_2019;

//pointer ke pasien berikutnya
Pasien_2511532019 next_2019;
```

- public class Pasien_2511532019 {
→Membuat class bernama Pasien_2511532019.
- String namaPasien_2019;
→Variabel untuk menyimpan nama pasien.
- String keluhan_2019;
→Variabel untuk menyimpan keluhan pasien.
- int nomorAntrian_2019;
→Variabel untuk menyimpan nomor antrian pasien.
- Pasien_2511532019 next_2019;
→Variabel pointer yang menunjuk ke pasien berikutnya pada linked list.

```
public Pasien_2511532019(String namaPasien_2019, String
keluhan_2019, int nomorAntrian_2019) {
    this.namaPasien_2019=namaPasien_2019;
    this.keluhan_2019=keluhan_2019;
    this.nomorAntrian_2019=nomorAntrian_2019;
    this.next_2019=null;
}
```

- public Pasien_2511532019(...) {
→Konstruktor untuk menginisialisasi data pasien.
- this.namaPasien_2019=namaPasien_2019;
→Mengisi atribut nama pasien.
- this.keluhan_2019=keluhan_2019;
→Mengisi atribut keluhan pasien.
- this.nomorAntrian_2019=nomorAntrian_2019;
→Mengisi atribut nomor antrian.
- this.next_2019=null;
→Pointer next diisi null karena belum terhubung ke node lain.

```
public String getNamaPasien_2019() {
    return namaPasien_2019;
}
public String getKeluhan_2019() {
```

```

        return keluhan_2019;
    }
    public int getNomorAntrian_2019() {
        return nomorAntrian_2019;
    }
    public Pasien_2511532019 getNext_2019() {
        return next_2019;
    }
}

```

- public String getNamaPasien_2019() {
→Method getter untuk mengambil nama pasien.
- return namaPasien_2019;
→Mengembalikan nilai nama pasien.
- public String getKeluhan_2019() {
→Method getter untuk mengambil keluhan pasien.
- return keluhan_2019;
→Mengembalikan nilai keluhan pasien.
- public int getNomorAntrian_2019() {
→Method getter untuk mengambil nomor antrian.
- return nomorAntrian_2019;
→Mengembalikan nomor antrian pasien.
- public Pasien_2511532019 getNext_2019() {
→Method getter untuk mengambil pointer next.
- return next_2019;
→Mengembalikan node berikutnya.

```

    public void setNamaPasien_2019(String namaPasien_2019) {
        this.namaPasien_2019=namaPasien_2019;
    }
    public void setKeluhan_2019(String keluhan_2019) {
        this.keluhan_2019=keluhan_2019;
    }
    public void setNomorAntrian_2019(int nomorAntrian_2019) {
        this.nomorAntrian_2019=nomorAntrian_2019;
    }
    public void setNext_2019(Pasien_2511532019 next_2019) {
        this.next_2019=next_2019;
    }
}

```

- public void setNamaPasien_2019(...) {
→Method setter untuk mengubah nama pasien.

- `this.namaPasien_2019=namaPasien_2019;`
→ Mengubah nilai nama pasien.
- `public void setKeluhan_2019(...) {`
→ Method setter untuk mengubah keluhan pasien.
- `this.keluhan_2019=keluhan_2019;`
→ Mengubah nilai keluhan pasien.
- `public void setNomorAntrian_2019(...) {`
→ Method setter untuk mengubah nomor antrian.
- `this.nomorAntrian_2019=nomorAntrian_2019;`
→ Mengubah nilai nomor antrian pasien.
- `public void setNext_2019(...) {`
→ Method setter untuk mengubah pointer next.
- `this.next_2019=next_2019;`
→ Pointer diarahkan ke node berikutnya.

Program Class RumahSakit

```
package pekan5_25115320119;

import java.util.Scanner;

public class RumahSakit_2511532019 {

    static Pasien_2511532019 head_2019=null;
    static int nomor_2019=0;

    public static void daftarPasien_2019(String nama_2019, String
keluhan_2019) {
        nomor_2019++;

        Pasien_2511532019 newPasien_2019=
            new Pasien_2511532019(nama_2019,
keluhan_2019, nomor_2019);

        //jika list kosong
        if(head_2019==null) {
            head_2019=newPasien_2019;
        }else {
            //telusuri ke node terakhir
            Pasien_2511532019 curr_2019=head_2019;

            while(curr_2019.next_2019!=null) {
                curr_2019=curr_2019.next_2019;
            }
        }
    }
}
```



```

        curr_2019.next_2019=newPasien_2019;
    }

    System.out.println("Pasien berhasil didaftarkan!");
    System.out.println("Nomor Antrian : "+nomor_2019);
}

public static void panggilPasien_2019() {
    //jika list kosong
    if(head_2019==null) {
        System.out.println("Antrian kosong");
        return;
    }

    System.out.println("Pasien dipanggil:");
    System.out.println("Nama      :
"+head_2019.namaPasien_2019);
    System.out.println("Keluhan   : "+head_2019.keluhan_2019);
    System.out.println("Antrian   :
"+head_2019.nomorAntrian_2019);

    head_2019=head_2019.next_2019;
}

//fungsi tampilkan antrian
public static void tampilAntrian_2019() {
    //jika list kosong
    if(head_2019==null) {
        System.out.println("Antrian kosong");
        return;
    }

    Pasien_2511532019 curr_2019=head_2019;

    System.out.println("Daftar Antrian Pasien:");

    while(curr_2019!=null) {
        System.out.println("Nomor Antrian :
"+curr_2019.nomorAntrian_2019);
        System.out.println("Nama Pasien   :
"+curr_2019.namaPasien_2019);
        System.out.println("Keluhan       :
"+curr_2019.keluhan_2019);

        curr_2019=curr_2019.next_2019;
    }
}

//fungsi cari pasien
public static void cariPasien_2019(String namaCari_2019) {
    //jika list kosong
    if(head_2019==null) {
        System.out.println("Antrian kosong");
        return;
    }

    Pasien_2511532019 curr_2019=head_2019;

```

```

        boolean ketemu_2019=false;

        while(curr_2019!=null) {

            if(curr_2019.namaPasien_2019.equalsIgnoreCase(namaCari_2019)) {
                System.out.println("Pasien ditemukan");
                System.out.println("Nomor Antrian :
"+curr_2019.nomorAntrian_2019);
                System.out.println("Nama Pasien :
"+curr_2019.namaPasien_2019);
                System.out.println("Keluhan :
"+curr_2019.keluhan_2019);

                ketemu_2019=true;
                break;
            }

            curr_2019=curr_2019.next_2019;
        }

        if(ketemu_2019==false) {
            System.out.println("Pasien tidak ditemukan");
        }
    }

    //fungsi cek status antrian
    public static void statusAntrian_2019() {
        //jika list kosong
        if(head_2019==null) {
            System.out.println("Antrian kosong");
            return;
        }

        int jumlah_2019=0;

        Pasien_2511532019 curr_2019=head_2019;

        while(curr_2019!=null) {
            jumlah_2019++;
            curr_2019=curr_2019.next_2019;
        }

        System.out.println("Jumlah pasien dalam antrian :
"+jumlah_2019);
        System.out.println("Pasien terdepan :
"+head_2019.namaPasien_2019);
    }

    public static void main(String[] args) {
        Scanner input_2019=new Scanner(System.in);

        int pilih_2019;

        do {
            System.out.println("\n=== Antrian Rumah Sakit
2511532019 ===");
            System.out.println("1. Daftarkan Pasien");

```

```

        System.out.println("2. Panggil Pasien");
        System.out.println("3. Tampilkan Antrian");
        System.out.println("4. Cari Pasien");
        System.out.println("5. Cek Status Antrian");
        System.out.println("6. Keluar");
        System.out.print("Pilihan : ");
        pilih_2019=input_2019.nextInt();
        input_2019.nextLine();

        switch(pilih_2019) {

        case 1:
            System.out.print("Masukkan Nama Pasien : ");
            String nama_2019=input_2019.nextLine();

            System.out.print("Masukkan Keluhan : ");
            String keluhan_2019=input_2019.nextLine();

            daftarPasien_2019(nama_2019, keluhan_2019);
            break;

        case 2:
            panggilPasien_2019();
            break;

        case 3:
            tampilAntrian_2019();
            break;

        case 4:
            System.out.print("Masukkan nama pasien yang
dicari : ");
            String cari_2019=input_2019.nextLine();

            cariPasien_2019(cari_2019);
            break;

        case 5:
            statusAntrian_2019();
            break;

        case 6:
            System.out.println("Program selesai");
            break;

        default:
            System.out.println("Pilihan tidak valid");
        }

    }while(pilih_2019!=6);

    input_2019.close();
}
}

```

```
import java.util.Scanner;

public class RumahSakit_2511532019 {

    static Pasien_2511532019 head_2019=null;
    static int nomor_2019=0;

    public static void daftarPasien_2019(String nama_2019, String
keluhan_2019) {
        nomor_2019++;

        Pasien_2511532019 newPasien_2019=
            new Pasien_2511532019(nama_2019,
keluhan_2019, nomor_2019);
```

- import java.util.Scanner;
→ Mengimpor class Scanner untuk input dari keyboard.
- public class RumahSakit_2511532019 {
→ Membuat class bernama RumahSakit_2511532019.
- static Pasien_2511532019 head_2019=null;
→ Variabel head_2019 digunakan sebagai head linked list dan awalnya bernilai null.
- static int nomor_2019=0;
→ Variabel counter untuk nomor antrian pasien.
- public static void daftarPasien_2019(...) {
→ Method untuk mendaftarkan pasien baru.
- nomor_2019++;
→ Nomor antrian bertambah otomatis setiap pasien baru masuk.
- Pasien_2511532019 newPasien_2019=
→ Membuat variabel node baru.
- new Pasien_2511532019(...)
→ Membuat object pasien baru dengan nama, keluhan, dan nomor antrian.

```
//jika list kosong
if(head_2019==null) {
    head_2019=newPasien_2019;
}else {
    //telusuri ke node terakhir
    Pasien_2511532019 curr_2019=head_2019;

    while(curr_2019.next_2019!=null) {
        curr_2019=curr_2019.next_2019;
    }
}
```

```

        curr_2019.next_2019=newPasien_2019;
    }

    System.out.println("Pasien berhasil didaftarkan!");
    System.out.println("Nomor Antrian : "+nomor_2019);
}

```

- if(head_2019==null) {
→Mengecek apakah linked list kosong.
- head_2019=newPasien_2019;
→Jika kosong, pasien baru menjadi head.
- }else {
→Jika list tidak kosong, masuk ke proses traversal.
- Pasien_2511532019 curr_2019=head_2019;
→Variabel curr_2019 mulai dari head.
- while(curr_2019.next_2019!=null) {
→Perulangan berjalan selama masih ada node berikutnya.
- curr_2019=curr_2019.next_2019;
→Pointer berpindah ke node berikutnya.
- curr_2019.next_2019=newPasien_2019;
→Node terakhir dihubungkan ke node pasien baru.
- System.out.println("Pasien berhasil didaftarkan!");
→Menampilkan pesan pasien berhasil ditambahkan.
- System.out.println("Nomor Antrian : "+nomor_2019);
→Menampilkan nomor antrian pasien.

```

public static void panggilPasien_2019() {
    //jika list kosong
    if(head_2019==null) {
        System.out.println("Antrian kosong");
        return;
    }

    System.out.println("Pasien dipanggil:");
    System.out.println("Nama      : 
"+head_2019.namaPasien_2019);
    System.out.println("Keluhan   : "+head_2019.keluhan_2019);
    System.out.println("Antrian   : 
"+head_2019.nomorAntrian_2019);

    head_2019=head_2019.next_2019;
}

```

- `public static void panggilPasien_2019() {`
→ Method untuk memanggil pasien pertama.
- `if(head_2019==null) {`
→ Mengecek apakah antrian kosong.
- `System.out.println("Antrian kosong");`
→ Menampilkan pesan antrian kosong.
- `return;`
→ Menghentikan method.
- `System.out.println("Pasien dipanggil:");`
→ Menampilkan teks pasien dipanggil.
- `System.out.println("Nama : "+head_2019.namaPasien_2019);`
→ Menampilkan nama pasien terdepan.
- `System.out.println("Keluhan : "+head_2019.keluhan_2019);`
→ Menampilkan keluhan pasien.
- `System.out.println("Antrian : "+head_2019.nomorAntrian_2019);`
→ Menampilkan nomor antrian pasien.
- `head_2019=head_2019.next_2019;`
→ Head dipindahkan ke node berikutnya sehingga pasien pertama terhapus.

```
//fungsi tampilkan antrian
public static void tampilAntrian_2019() {
    //jika list kosong
    if(head_2019==null) {
        System.out.println("Antrian kosong");
        return;
    }

    Pasien_2511532019 curr_2019=head_2019;

    System.out.println("Daftar Antrian Pasien:");

    while(curr_2019!=null) {
        System.out.println("Nomor Antrian :
"+curr_2019.nomorAntrian_2019);
        System.out.println("Nama Pasien :
"+curr_2019.namaPasien_2019);
        System.out.println("Keluhan :
"+curr_2019.keluhan_2019);

        curr_2019=curr_2019.next_2019;
    }
}
```

- `public static void tampilAntrian_2019() {`
→ Method untuk menampilkan seluruh antrian.
- `if(head_2019==null) {`
→ Mengecek apakah linked list kosong.
- `System.out.println("Antrian kosong");`
→ Menampilkan pesan antrian kosong.
- `return;`
→ Menghentikan method.
- `Pasien_2511532019 curr_2019=head_2019;`
→ Variabel traversal mulai dari head.
- `System.out.println("Daftar Antrian Pasien:");`
→ Menampilkan judul daftar antrian.
- `while(curr_2019!=null) {`
→ Perulangan berjalan selama node masih ada.
- `System.out.println("Nomor Antrian : "+curr_2019.nomorAntrian_2019);`
→ Menampilkan nomor antrian pasien.
- `System.out.println("Nama Pasien : "+curr_2019.namaPasien_2019);`
→ Menampilkan nama pasien.
- `System.out.println("Keluhan : "+curr_2019.keluhan_2019);`
→ Menampilkan keluhan pasien.
- `curr_2019=curr_2019.next_2019;`
Pointer berpindah ke node berikutnya.

```
//fungsi cari pasien
public static void cariPasien_2019(String namaCari_2019) {
    //jika list kosong
    if(head_2019==null) {
        System.out.println("Antrian kosong");
        return;
    }
}
```

- `public static void cariPasien_2019(...) {`
→ Method untuk mencari pasien.
- `if(head_2019==null) {`
→ Mengecek apakah antrian kosong.

- `System.out.println("Antrian kosong");`
→ Menampilkan pesan antrian kosong.
- `return;`
→ Menghentikan method.

```

        Pasien_2511532019 curr_2019=head_2019;
        boolean ketemu_2019=false;

        while(curr_2019!=null) {

            if(curr_2019.namaPasien_2019.equalsIgnoreCase(namaCari_2019)) {
                System.out.println("Pasien ditemukan");
                System.out.println("Nomor Antrian :
"+curr_2019.nomorAntrian_2019);
                System.out.println("Nama Pasien :
"+curr_2019.namaPasien_2019);
                System.out.println("Keluhan :
"+curr_2019.keluhan_2019);

                ketemu_2019=true;
                break;
            }

            curr_2019=curr_2019.next_2019;
        }

        if(ketemu_2019==false) {
            System.out.println("Pasien tidak ditemukan");
        }
    }

```

- `Pasien_2511532019 curr_2019=head_2019;`
→ Variabel traversal mulai dari head.
- `boolean ketemu_2019=false;`
→ Variabel penanda apakah pasien ditemukan.
- `while(curr_2019!=null) {`
→ Perulangan traversal linked list.
- `if(curr_2019.namaPasien_2019.equalsIgnoreCase(namaCari_2019)) {`
→ Mengecek nama pasien tanpa membedakan huruf besar kecil.
- `System.out.println("Pasien ditemukan");`
→ Menampilkan pesan pasien ditemukan.
- `System.out.println("Nomor Antrian : "+curr_2019.nomorAntrian_2019);`
→ Menampilkan nomor antrian pasien.

- `System.out.println("Nama Pasien : "+curr_2019.namaPasien_2019);`
→ Menampilkan nama pasien.
- `System.out.println("Keluhan : "+curr_2019.keluhan_2019);`
→ Menampilkan keluhan pasien.
- `ketemu_2019=true;`
→ Menandakan pasien berhasil ditemukan.
- `break;`
→ Menghentikan perulangan.
- `curr_2019=curr_2019.next_2019;`
→ Pointer berpindah ke node berikutnya.
- `if(ketemu_2019==false) {`
→ Mengecek apakah pasien tidak ditemukan.
- `System.out.println("Pasien tidak ditemukan");`
→ Menampilkan pesan pasien tidak ditemukan

```
//fungsi cek status antrian
public static void statusAntrian_2019() {
    //jika list kosong
    if(head_2019==null) {
        System.out.println("Antrian kosong");
        return;
    }

    int jumlah_2019=0;

    Pasien_2511532019 curr_2019=head_2019;

    while(curr_2019!=null) {
        jumlah_2019++;
        curr_2019=curr_2019.next_2019;
    }

    System.out.println("Jumlah pasien dalam antrian :
    "+jumlah_2019);
    System.out.println("Pasien terdepan :
    "+head_2019.namaPasien_2019);
}
```

- `public static void statusAntrian_2019() {`
→ Method untuk mengecek status antrian.
- `if(head_2019==null) {`
→ Mengecek apakah antrian kosong.

- `System.out.println("Antrian kosong");`
→ Menampilkan pesan antrian kosong.
- `return;`
→ Menghentikan method.
- `int jumlah_2019=0;`
→ Variabel untuk menghitung jumlah pasien.
- `Pasien_2511532019 curr_2019=head_2019;`
→ Variabel traversal mulai dari head.
- `while(curr_2019!=null) {`
→ Perulangan traversal linked list.
- `jumlah_2019++;`
→ Menambah jumlah pasien.
- `curr_2019=curr_2019.next_2019;`
→ Pointer berpindah ke node berikutnya.
- `System.out.println("Jumlah pasien dalam antrian : "+jumlah_2019);`
→ Menampilkan jumlah pasien.
- `System.out.println("Pasien terdepan : "+head_2019.namaPasien_2019);`
→ Menampilkan nama pasien terdepan.

```
public static void main(String[] args) {
    Scanner input_2019=new Scanner(System.in);

    int pilih_2019;

    do {
        System.out.println("\n=== Antrian Rumah Sakit
2511532019 ===");
        System.out.println("1. Daftarkan Pasien");
        System.out.println("2. Panggil Pasien");
        System.out.println("3. Tampilkan Antrian");
        System.out.println("4. Cari Pasien");
        System.out.println("5. Cek Status Antrian");
        System.out.println("6. Keluar");
        System.out.print("Pilihan : ");
        pilih_2019=input_2019.nextInt();
        input_2019.nextLine();
    } while (pilih_2019 < 6);
}
```

- `public static void main(String[] args) {`
→ Method utama program.

- `Scanner input_2019=new Scanner(System.in);`
→ Membuat object Scanner untuk input keyboard.
- `int pilih_2019;`
→ Variabel untuk menyimpan pilihan menu.
- `do {`
→ Memulai perulangan menu.
- `System.out.println("\n=== Antrian Rumah Sakit 2511532019 ===");`
→ Menampilkan judul program.
- `System.out.println("1. Daftarkan Pasien");`
→ Menampilkan menu daftar pasien.
- `System.out.println("2. Panggil Pasien");`
→ Menampilkan menu panggil pasien.
- `System.out.println("3. Tampilkan Antrian");`
→ Menampilkan menu tampil antrian.
- `System.out.println("4. Cari Pasien");`
→ Menampilkan menu cari pasien.
- `System.out.println("5. Cek Status Antrian");`
→ Menampilkan menu status antrian.
- `System.out.println("6. Keluar");`
→ Menampilkan menu keluar program.
- `System.out.print("Pilihan : ");`
→ Meminta input pilihan menu.
- `pilih_2019=input_2019.nextInt();`
→ Menyimpan pilihan user.
- `input_2019.nextLine();`
→ dipakai untuk membersihkan Enter tersebut supaya input berikutnya tidak terlewat.

```
switch(pilih_2019) {
    case 1:
        System.out.print("Masukkan Nama Pasien : ");
        String nama_2019=input_2019.nextLine();

        System.out.print("Masukkan Keluhan : ");
        String keluhan_2019=input_2019.nextLine();
```

```
daftarPasien_2019(nama_2019, keluhan_2019);  
break;
```

- switch(pilih_2019) {
→Memilih proses berdasarkan input menu.
- case 1:
→Kondisi menu nomor 1.
- System.out.print("Masukkan Nama Pasien : ");
→Meminta input nama pasien.
- String nama_2019=input_2019.nextLine();
→Menyimpan nama pasien.
- System.out.print("Masukkan Keluhan : ");
→Meminta input keluhan pasien.
- String keluhan_2019=input_2019.nextLine();
→Menyimpan keluhan pasien.
- daftarPasien_2019(nama_2019, keluhan_2019);
→Memanggil method untuk menambahkan pasien.
- break;
→Menghentikan case 1.

```
case 2:  
    panggilPasien_2019();  
break;
```

- case 2:
→Kondisi menu nomor 2.
- panggilPasien_2019();
→Memanggil method untuk memanggil pasien.
- break;
→Menghentikan case 2.

```
case 3:  
    tampilAntrian_2019();  
break;
```

- case 3:
→Kondisi menu nomor 3.

- tampilAntrian_2019();
→Memanggil method untuk menampilkan antrian.
- break;
→Menghentikan case 3.

```

        case 4:
            System.out.print("Masukkan nama pasien yang
dicari : ");
            String cari_2019=input_2019.nextLine();

            cariPasien_2019(cari_2019);
            break;

```

- case 4:
→Kondisi menu nomor 4.
- System.out.print("Masukkan nama pasien yang dicari : ");
→Meminta nama pasien yang ingin dicari.
- String cari_2019=input_2019.nextLine();
→Menyimpan nama pasien yang dicari.
- cariPasien_2019(cari_2019);
→Memanggil method pencarian pasien.
- break;
→Menghentikan case 4.

```

        case 5:
            statusAntrian_2019();
            break;

```

- case 5:
→Kondisi menu nomor 5.
- statusAntrian_2019();
→Memanggil method status antrian.
- break;
→Menghentikan case 5.

```

        case 6:
            System.out.println("Program selesai");
            break;

```

- case 6:
→Kondisi menu nomor 6.

- `System.out.println("Program selesai");`
→ Menampilkan pesan program selesai.
- `break;`
→ Menghentikan case 6.

```
default:
    System.out.println("Pilihan tidak valid");
}
```

- `default:`
→ Kondisi jika pilihan menu tidak tersedia.
- `System.out.println("Pilihan tidak valid");`
→ Menampilkan pesan error pilihan salah.

```
}while(pilih_2019!=6);
    input_2019.close();
}
```

- `}while(pilih_2019!=6);`
Perulangan akan terus berjalan selama pilihan bukan 6.
- `input_2019.close();`
Menutup object Scanner.

Output

```
=== Antrian Rumah Sakit 2511532019 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan : 1
Masukkan Nama Pasien : budisantoso
Masukkan Keluhan : demam
Pasien berhasil didaftarkan!
Nomor Antrian : 1
```

```
=== Antrian Rumah Sakit 2511532019 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan : 1
Masukkan Nama Pasien : udin
Masukkan Keluhan : sakit perut
Pasien berhasil didaftarkan!
Nomor Antrian : 2
```

```
=== Antrian Rumah Sakit 2511532019 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan : 3
Daftar Antrian Pasien:
Nomor Antrian : 1
Nama Pasien : budisantoso
Keluhan : demam
Nomor Antrian : 2
Nama Pasien : udin
Keluhan : sakit perut
```

```
=== Antrian Rumah Sakit 2511532019 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan : 2
Pasien dipanggil:
Nama : budisantoso
Keluhan : demam
Antrian : 1
```

```

=== Antrian Rumah Sakit 2511532019 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan : 3
Daftar Antrian Pasien:
Nomor Antrian : 2
Nama Pasien   : udin
Keluhan       : sakit perut

=== Antrian Rumah Sakit 2511532019 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan : 1
Masukkan Nama Pasien : pujiastuti
Masukkan Keluhan : flu
Pasien berhasil didaftarkan!
Nomor Antrian : 3

```

```

=== Antrian Rumah Sakit 2511532019 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan : 3
Daftar Antrian Pasien:
Nomor Antrian : 2
Nama Pasien   : udin
Keluhan       : sakit perut
Nomor Antrian : 3
Nama Pasien   : pujiastuti
Keluhan       : flu

=== Antrian Rumah Sakit 2511532019 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan : 4
Masukkan nama pasien yang dicari : pujiastuti
Pasien ditemukan
Nomor Antrian : 3
Nama Pasien   : pujiastuti
Keluhan       : flu

```



```
=== Antrian Rumah Sakit 2511532019 ===  
1. Daftarkan Pasien  
2. Panggil Pasien  
3. Tampilkan Antrian  
4. Cari Pasien  
5. Cek Status Antrian  
6. Keluar  
Pilihan : 5  
Jumlah pasien dalam antrian : 2  
Pasien terdepan : udin  
  
=== Antrian Rumah Sakit 2511532019 ===  
1. Daftarkan Pasien  
2. Panggil Pasien  
3. Tampilkan Antrian  
4. Cari Pasien  
5. Cek Status Antrian  
6. Keluar  
Pilihan : 6  
Program selesai
```